

INFORME DE AVANCES DEL PROYECTO DE ELECTIVA 3

**ESTUDIANTES: VANYA CATALINA PORTILLA SANCHEZ,
FRANKLIN SNEIDER CORDOBA D ELA CRUZ
ANDRES ALIRIO BURBANO SOLARTE
JAIDER CHINDOY
JHONATAN MAURICIO MUCHAVISoy JAJoy**

DOCENTE: KEVIN RODRIGUEZ

**INSTITUTO UNIVERSITARIO DEL PUTUMAYO
ELECTIVA 3
SEXTO SEMESTRE
MOCOA – PUTUMAYO
2026**

Backend (NestJS)

Introducción

El backend de este proyecto está desarrollado con NestJS y TypeScript, siguiendo una arquitectura modular que facilita la escalabilidad y el mantenimiento. Su propósito es proveer servicios y APIs para la gestión de usuarios, productos, inventario y autenticación.

Objetivos

- Implementar un sistema seguro y modular para la administración de datos.
- Proveer endpoints REST que permitan la interacción con el frontend.
- Garantizar la integridad y consistencia de la información mediante validaciones y reglas de negocio.

Metodología

Se utilizó el framework NestJS por su enfoque en la arquitectura modular y su compatibilidad con TypeScript. La base de datos se configuró en el módulo database, y se implementaron módulos específicos (auth, usuarios, productos, inventario, categoria) para separar responsabilidades.

Arquitectura

- Controladores: Definen las rutas y reciben las solicitudes HTTP.
- Servicios: Contienen la lógica de negocio.
- Módulos: Agrupan controladores y servicios relacionados.
- Archivos principales: app.module.ts, main.ts, seed.service.ts.

Dependencias principales

- NestJS
- TypeORM/Mongoose (según la base de datos usada)
- dotenv (manejo de variables de entorno)
- JWT (autenticación segura)

Configuración

- .env: variables de entorno (puerto, credenciales DB, claves JWT).
- database/: configuración de conexión a la base de datos.
- auth/: manejo de tokens y seguridad.

Endpoints principales

- POST /auth/login → autenticación de usuarios.
- GET /usuarios → listado de usuarios.
- POST /productos → creación de productos.

- GET /inventario → consulta de inventario.

Ejecución

```
Bash ^  
  
npm install  
npm run start:dev
```

Pruebas

```
Bash ^  
  
npm run test
```

Frontend (Vite + Tailwind + TypeScript)

El frontend está desarrollado con Vite, TypeScript y Tailwind CSS, proporcionando una interfaz moderna y eficiente para interactuar con el backend.

Objetivos

- Ofrecer una experiencia de usuario intuitiva y rápida.
- Consumir los servicios del backend mediante llamadas API.
- Implementar componentes reutilizables y estilizados con Tailwind.

Metodología

Se utilizó Vite por su rapidez en el desarrollo y compilación. Los estilos se gestionan con Tailwind CSS, y la comunicación con el backend se realiza mediante Axios/Fetch.

Arquitectura

- Carpeta src/: contiene los componentes principales.
- Carpeta public/: recursos estáticos.
- Archivos de configuración: vite.config.ts, tailwind.config.js, tsconfig.json.

Dependencias principales

- React (si está implementado).
- Tailwind CSS.
- Axios/Fetch.
- ESLint.

Configuración

- .env: define la URL del backend.
- vite.config.ts: configuración del entorno de desarrollo.

Componentes principales

- Login: autenticación de usuarios.
- Dashboard: vista principal.
- Productos: gestión de productos.
- Usuarios: administración de usuarios.

Ejecución

Bash ^

```
npm install  
npm run dev
```

Pruebas

Bash ^

```
npm run test
```

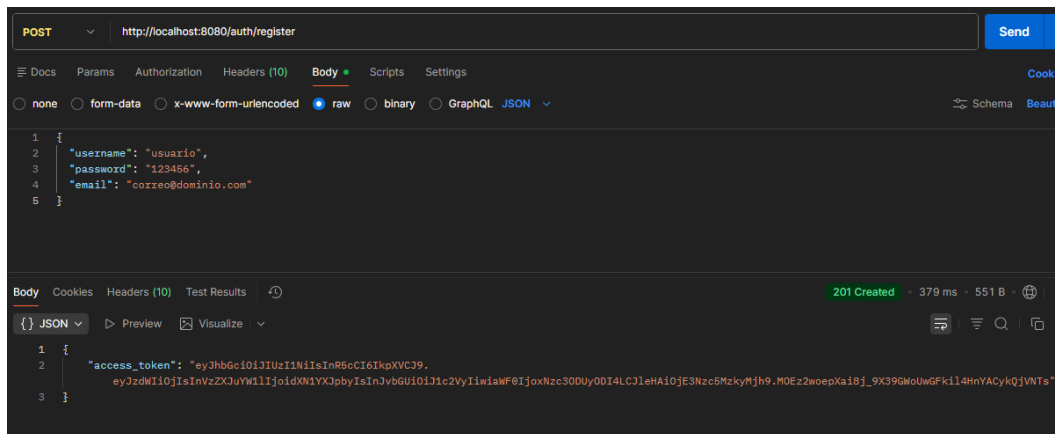
Conclusiones

El backend cumple con los principios de modularidad y seguridad, garantizando una base sólida para aplicaciones escalables y mantenibles.

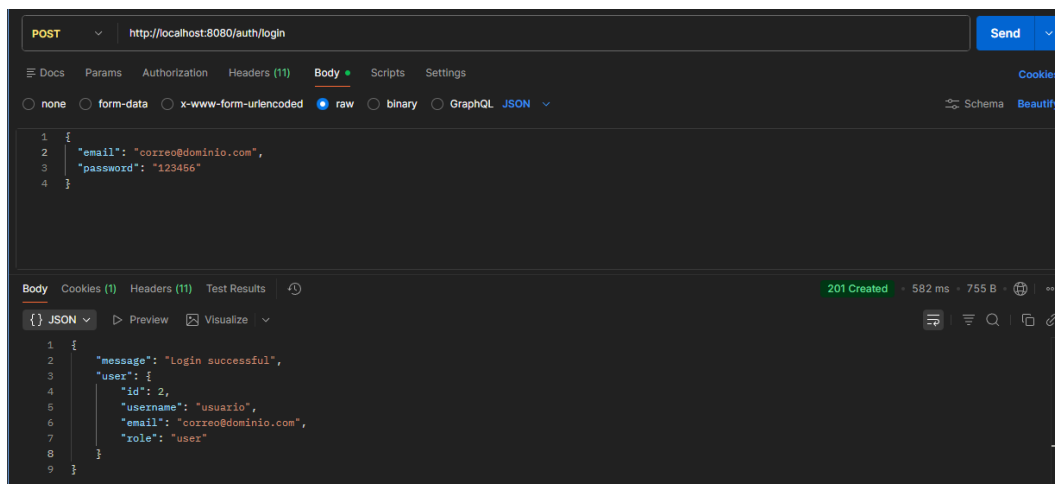
El frontend ofrece una interfaz clara y eficiente, conectándose de manera óptima con el backend y garantizando una experiencia de usuario fluida.

1. Autenticación

- [POST /auth/register](#)

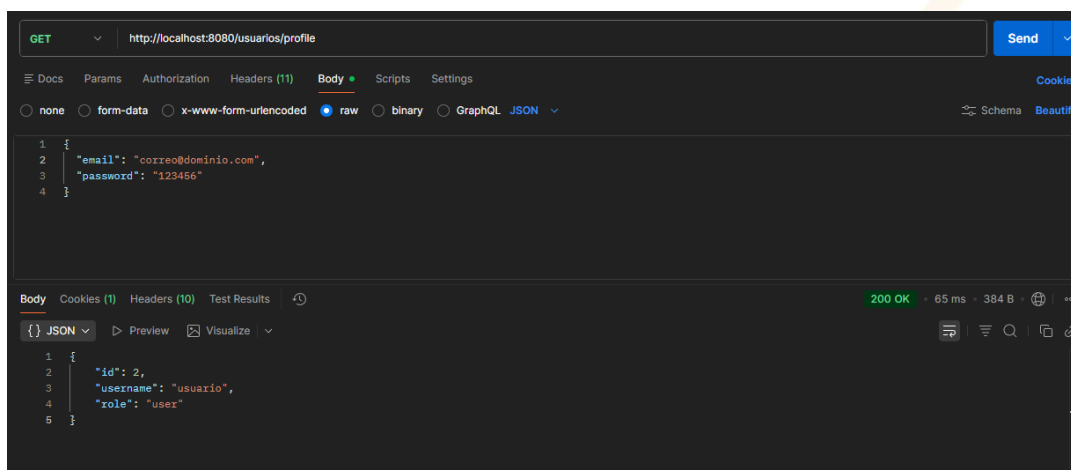


- POST /auth/login



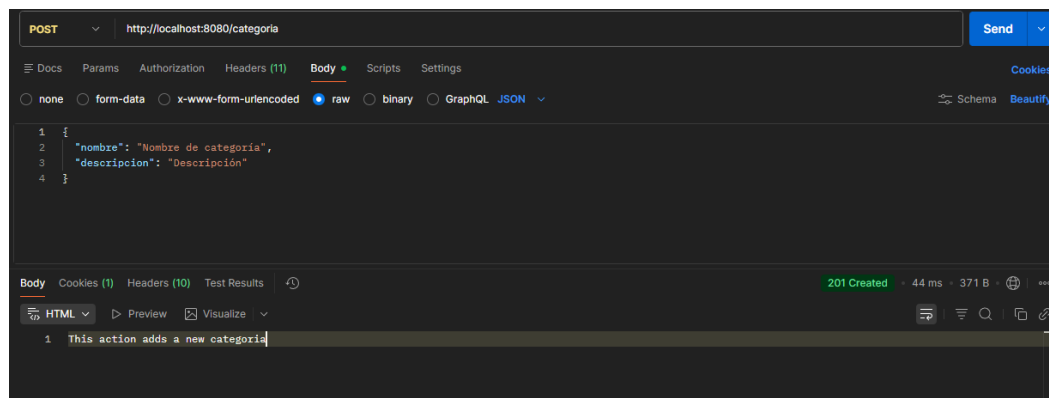
2. Usuarios

- GET /usuarios/profile



3. Categoría

- POST /categoria



- GET /categoria/1

